ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO BÀI TẬP E2

Học sâu và ứng dụng trong thị giác máy tính — CO5085

Học kỳ	HK261 (2026–2027)
Giảng viên	Lê Thành Sách, Nguyễn Quốc Minh
Sinh viên	Dương Gia An — MSSV 2470293
Hình thức	Cá nhân
Dữ liệu	MNIST (28×28, 10 lớp)
Ngày hoàn thành	05/11/2026

Mục lục

1. Giới thiệu và thiết lập chung **3**

2. E2 — Transformer: MSA và tokenization **3**

2.1. Multi-Head Self-Attention 3

2.2. Các cách lấy token 3

2.3. Kết quả 4

3. Kết luận **5**

4. Phụ lục: tái lập **6**

Khai báo sử dụng công cụ AI 6

1. Giới thiệu và thiết lập chung

Báo cáo trình bày bài tập E2 (MSA và tokenization) trên tập dữ liệu nhỏ MNIST: 60.000 ảnh huấn luyện (chia 50.000 train / 10.000 validation theo seed 42) và 10.000 ảnh test chính thức. Split này dùng chung cho cả E1, E2 và E3 nên mọi so sánh — kể cả giữa các họ mô hình khác nhau — là công bằng.

Ảnh được chuẩn hoá với mean 0,1307 và std 0,3081 (số liệu chuẩn của MNIST) và **không** dùng augmentation, để chênh lệch kết quả phản ánh kiến trúc chứ không phải may mắn của phép biến đổi dữ liệu. Mọi mô hình huấn luyện bằng vòng lặp viết tay (forward → loss → backward → optimizer.step), optimizer Adam, batch 128, và lưu lại *metrics.json* cùng *history.csv* cho từng lần chạy.

Thành phần	Giá trị
Dữ liệu	MNIST 28×28, 1 kênh, 10 lớp
Split	50.000 train / 10.000 val / 10.000 test (seed 42)
Tiền xử lý	chuẩn hoá mean=0,1307 std=0,3081; không augmentation
Optimizer / loss	Adam; CrossEntropyLoss trên logits
Batch / seed	128 / 42
Phần cứng	Intel i5-9300H, NVIDIA GTX 1650 4 GB, PyTorch 2.5.1

Bảng 4. Thiết lập thí nghiệm dùng chung cho E1–E3.

2. E2 — Transformer: MSA và tokenization

2.1. Multi-Head Self-Attention

MSA được viết lại từ các phép toán cơ bản (Linear, einsum, softmax, ghép head), không dùng *nn.MultiheadAttention*. Với đầu vào $X \in \mathbb{R}^{B\times N\times d}$: chiếu ra Q, K, V; tính điểm tương đồng đã scale; softmax theo trục key; lấy tổng có trọng số của V; ghép H head rồi chiếu ra output:

$$Q = XW_Q,\ K = XW_K,\ V = XW_V\ ;\ \text{head}_h = \text{softmax}(Q_h K_h^T/\sqrt{d_k})V_h\ ;\ \text{MSA}(X) = \text{concat}(\text{head}_{1..H})W_O$$

Hệ số $1/\sqrt{d_k}$ giữ phương sai của tích vô hướng bằng 1: nếu không, với d_k lớn, softmax rơi vào vùng bão hoà, gradient gần 0 và mô hình không học được. Đây cũng là lỗi hay gặp nhất khi tự viết.

d	Số head	N	Sai khác output	Sai khác attention	Có mask
128	4	50	8.9e-08	3.0e-08	8.9e-08
64	2	29	7.5e-08	3.7e-08	8.9e-08
96	3	16	1.0e-07	4.5e-08	8.9e-08

Bảng 5. Kiểm chứng MSA tự viết khớp nn.MultiheadAttention khi copy trọng số.

Với cùng bộ trọng số, hai bản cho cùng output với sai khác lớn nhất **1.0e-07** — sai số số học thuần tuý của phép cộng dấu phẩy động. Kết quả này cả khi bật mask, nên có thể khẳng định bản tự viết là đúng đặc tả.

2.2. Các cách lấy token

Ảnh 28×28 không có token tự nhiên như câu chữ, nên cách chia token là một lựa chọn thiết kế có ảnh hưởng trực tiếp: càng nhiều token thì chi phí attention $O(N^2 d)$ càng lớn, nhưng mỗi token chứa ít thông tin hơn. Ba cách được triển khai: chia lưới patch 4×4 (49 token, mỗi token 16 chiều), lấy mỗi hàng ảnh làm một token (28 token, 28 chiều), và dùng CNN stem rút feature map 7×7 (49 token, 64 chiều). Mỗi chuỗi được bọc thêm một token CLS và một positional embedding học được, vì bản thân attention không có khái niệm thứ tự.

2.3. Kết quả

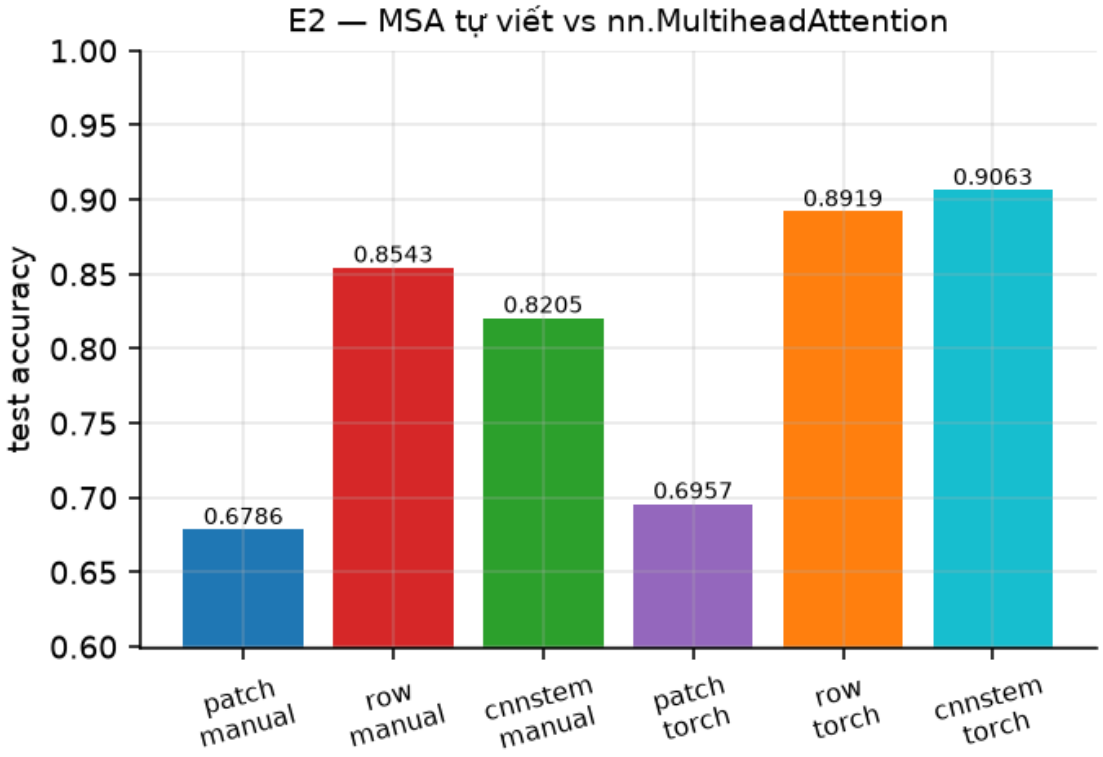
Tokenize	MSA	Số token	Tham số	Test acc	s/epoch
patch	manual	50	275,210	67.86%	17.4
patch	torch	50	275,210	69.57%	17.8
row	manual	29	274,058	85.43%	21.1
row	torch	29	274,058	89.19%	14.3
cnnstem	manual	50	300,266	82.05%	20.2
cnnstem	torch	50	300,266	90.63%	21.1

Bảng 6. Kết quả E2: 3 cách tokenize × 2 bản MSA (dim 128, depth 2, 4 head, 4 epoch).

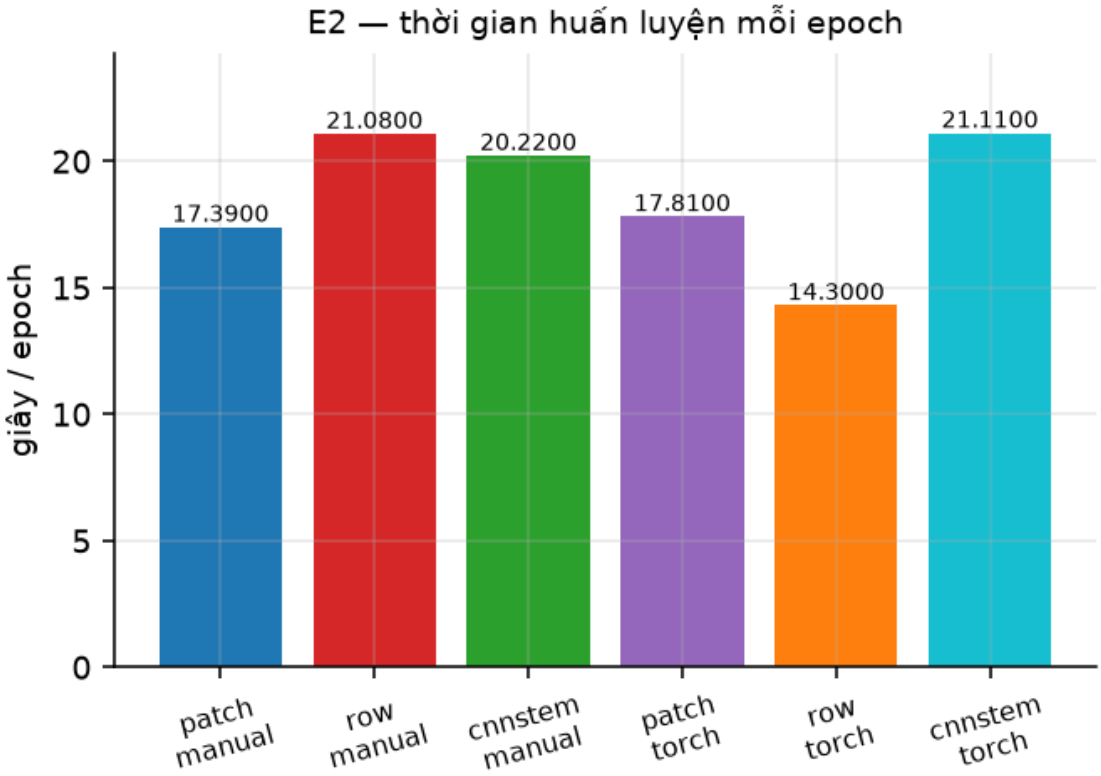
Bản tự viết đạt trung bình 78.45% so với 83.13% của nn.MultiheadAttention. Chênh lệch 4.7 điểm này **không phải** sai lệch toán học — Bảng 4 đã cho thấy hai bản trùng khớp tới 10^{-7} khi dùng chung trọng số. Nó đến từ khởi tạo khác nhau: bản PyTorch gộp ba phép chiếu thành một tensor nên thứ tự sinh số ngẫu nhiên khác bản tự viết, và với chỉ 4 epoch thì mức dao động đó lên tới vài điểm. Về tốc độ, bản tự viết chậm hơn (19.6 so với 17.7 s/epoch) do ba phép chiếu riêng thay vì một phép gộp và không dùng kernel tối ưu.

Ba cách tokenize cho kết quả **rất khác nhau**: patch 68.71%, row 87.31%, cnnstem 86.34% — chênh lệch tới 18.6 điểm phần trăm, lớn hơn nhiều so với chênh lệch do đổi bản MSA (4.7 điểm). Nguyên nhân nằm ở độ sâu: encoder chỉ có 2 block, nghĩa là mỗi token chỉ được trộn thông tin với các token khác hai lần. Với patch 4×4, mỗi token là 16 pixel thô và mô hình phải vừa học đặc trưng cục bộ vừa học cách tổ hợp — quá nhiều việc cho hai lớp attention. Token theo hàng đã chứa sẵn 28 điểm ảnh có cấu trúc nên dễ hơn; CNN stem đưa vào chuỗi những đặc trưng đã tích chập (trường tiếp nhận 5×5) nên đạt kết quả cao nhất. Kết luận: với encoder nông, phần lớn việc trích đặc trưng phải do tokenizer đảm nhiệm.

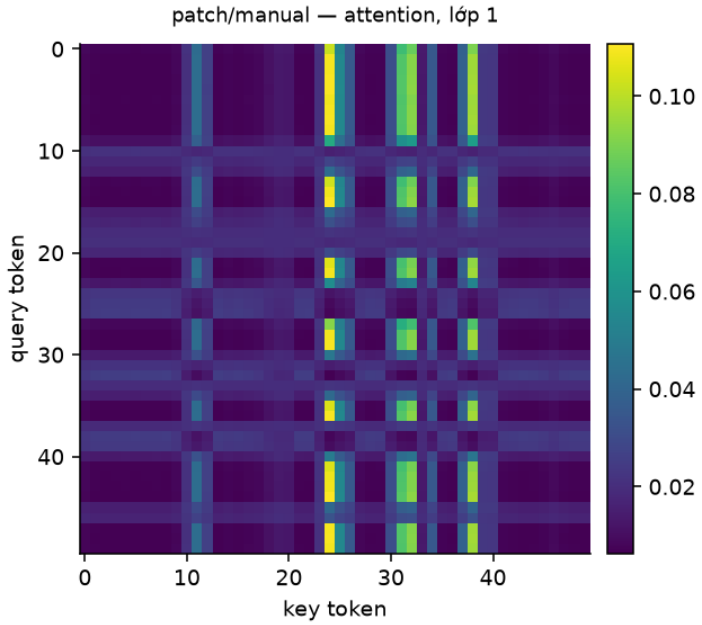
Kiểm tra thêm (tái lập được bằng `--epochs 12`): patch/manual huấn luyện 12 epoch chỉ đạt 69,9% test — tức mô hình bị chặn bởi độ sâu chứ không phải ngân sách epoch. Đây cũng là lý do các ViT thực tế dùng depth 12–24 thay vì 2.



Hình 6. Test accuracy: MSA tự viết vs nn.MultiheadAttention theo từng cách tokenize.



Hình 7. Thời gian huấn luyện mỗi epoch của hai bản MSA.



Hình 8. Bản đồ attention của block đầu (head có phân bố nhọn nhất) với tokenize patch: một vài patch giữ vai trò "key" chính, các patch còn lại gần như bị bỏ qua.

3. Kết luận

• **Tự viết attention là kiểm chứng được.** Bản MSA viết tay cho output trùng với nn.MultiheadAttention tới mức sai số dấu phẩy động khi dùng chung trọng số — tiêu chuẩn đúng đắn thay cho việc chỉ so accuracy.

• **Tokenization là lựa chọn thiết kế có giá.** Với encoder chỉ 2 block, ba cách chia token lệch nhau tới hàng chục điểm phần trăm: tokenizer nào làm sẵn nhiều việc trích đặc trưng hơn thì kết quả tốt hơn. Số

token đồng thời quyết định chi phí $O(N^2d)$.

• **Chi phí tính toán của bản tự viết cao hơn.** Ba phép chiếu rời thay vì một phép gộp khiến mỗi epoch chậm hơn bản PyTorch, dù chất lượng tương đương.

4. Phụ lục: tái lập

Kết quả trong báo cáo sinh lại được bằng các lệnh sau, mỗi lệnh ghi kết quả vào thư mục *results/* tương ứng:

```
python E2/run_all.py
python E2/test_msa.py
python reports/build_report.py
```

Mỗi lần chạy lưu *metrics.json* (cấu hình, số tham số, accuracy, thời gian), *history.csv* (loss/accuracy từng epoch) và các hình PNG. Seed cố định 42.

Khai báo sử dụng công cụ AI

Theo mục 1.6 của đề, tôi khai báo phạm vi đã sử dụng công cụ AI hỗ trợ trong bài tập này: đọc và tóm tắt đề; gợi ý bố cục thư mục và các API PyTorch phù hợp; soạn bản nháp ban đầu cho phần lớn mã nguồn trong *common/*, *E1/–E3/*, *A1/*, *A2/*; gợi ý dàn ý và cách trình bày báo cáo; gợi ý cách chẩn đoán một số lỗi shape và lỗi huấn luyện.

Phần tôi trực tiếp thực hiện: cài đặt môi trường và tải toàn bộ dữ liệu; chạy toàn bộ thí nghiệm trên máy của mình; kiểm tra sự nhất quán của từng bảng số và từng hình trong báo cáo với *metrics.json* mà các script sinh ra.